

SAPPHIRE LANGUAGE SPECIFICATION AND AUTOMATION MANUAL

Formal specification of Sapphire v1.0.0 language semantics, type system, standard library APIs, and the ml.distributed frontier training engine.

1. Language Primitives

Types: string, int, float, bool, list, dict, nil  
Keywords: let, fn, if, else, for, in, while, return, parallel, import, print  
Operators: + - \* / % == != < > <= >= && || ! |>

2. Standard Library Namespaces

| Namespace      | Key APIs                                                                                       |
|----------------|------------------------------------------------------------------------------------------------|
| os             | system_info(), notify(), clip_write(), clip_read(), sleep()                                    |
| fs             | read(), write(), exists(), delete(), list_dir(), copy()                                        |
| http           | get(), post(), put(), delete(), download()                                                     |
| ai             | prompt(), classify(), extract_json(), embed(), summarize()                                     |
| ml             | tensor(), randn(), rand(), zeros(), matmul(), autograd, model, train, gpu, kernel, distributed |
| ml.distributed | Transformer(), Cluster(), train() -- 5D frontier LLM engine                                    |
| agent          | memory.remember(), .recall(), autonomy.run_loop(), tools.register()                            |
| scheduler      | interval(), once(), cron(), cancel()                                                           |
| gui            | alert(), dialog(), window(), progress_bar()                                                    |
| data           | json.parse(), json.stringify(), csv.read(), yaml.parse()                                       |

3. ml.distributed.Transformer(config) -- Parameters

| Parameter   | Default | Description                          |
|-------------|---------|--------------------------------------|
| layers      | 32      | Number of transformer decoder layers |
| hidden      | 4096    | Hidden dimension size (d_model)      |
| heads       | 32      | Number of attention heads            |
| kv_heads    | None    | GQA: number of KV heads (None = MHA) |
| ff_mult     | 4       | FFN size = ff_mult * hidden (SwiGLU) |
| vocab       | 32000   | Vocabulary size                      |
| seq_len     | 4096    | Maximum sequence length              |
| precision   | bf16    | Training precision: bf16 or fp8      |
| moe_experts | 0       | MoE: number of experts (0 = dense)   |
| top_k       | 2       | MoE: top-k expert routing            |

4. ml.distributed.train() -- Result Fields

| Field                    | Description                                          |
|--------------------------|------------------------------------------------------|
| result.strategy          | Strategy string e.g. TP1_PP1_DP512_EP1_SP1_ZeRO3_FP8 |
| result.mfu_percent       | Model FLOPs Utilization in %                         |
| result.tokens_per_sec    | Training throughput in tokens/second                 |
| result.eta_days          | Estimated days to complete total token budget        |
| result.memory_per_gpu_gb | Peak memory used per GPU in GB                       |
| result.nccl_latency_ms   | Estimated AllReduce latency in milliseconds          |

|                     |                                                  |
|---------------------|--------------------------------------------------|
| result.codegen_path | Path to generated PyTorch FSDP + torchrun script |
|---------------------|--------------------------------------------------|

5. Kernel Dispatch Table (H100-80GB)

| Operation             | Kernel                   | Notes                                       |
|-----------------------|--------------------------|---------------------------------------------|
| Attention (seq<=8192) | FlashAttention-3         | SRAM-only, IO-optimal, H100/H200 only       |
| Attention (seq>8192)  | Triton Ring Attention    | Distributed long-sequence attention         |
| MLP SwiGLU            | Triton Fused SwiGLU      | Fused gate + MLP in single kernel           |
| LayerNorm/RMSNorm     | Triton Fused Norm        | Backward + forward in one pass              |
| RoPE Embeddings       | CUDA Fused RoPE          | In-place position encoding                  |
| Matmul (FP8)          | TransformerEngine cublas | E4M3/E5M2 with BF16 accumulation            |
| Matmul (BF16)         | cuBLAS Tensor Cores      | Standard BF16 GEMM                          |
| AllReduce             | NCCL Ring                | Topology-aware, async overlap with backward |

6. 5D Auto-Parallelism Solver Algorithm

Grid search over: TP in {1,2,4,8} x PP in {1,2,4,8} x DP = total\_gpus/(TP\*PP)  
x EP in {1,4,8,64} x SP in {1,2,4} x ZeRO in {1,2,3}

- Feasibility filters:
- 1. TP\*PP\*DP must equal num\_gpus
  - 2. Per-GPU memory (weights+grads+optimizer+activations) <= gpu\_vram\*0.95
  - 3. PP>1 requires layers % PP == 0
  - 4. TP>1 requires heads % TP == 0

Objective: maximize MFU = compute\_flops / (wall\_time \* peak\_flops \* num\_gpus)  
NCCL AllReduce latency is subtracted from compute time in the objective.